By moving this logic into the resource filter, you can remove it from your action methods and instead decorate the whole API controller with a simple attribute:

```
[FeatureEnabled(IsEnabled = true)]
[Route("api/recipe")]
public class RecipeApiController : ControllerBase
```

You've extracted only two lines of code from your action methods so far, but you're on the right track. In the next section we'll move on to action filters and extract two more filters from the action method code.

22.1.3 Action filters: Customizing model binding and action results

Action filters run just after model binding, before the action method executes. Thanks to this positioning, action filters can access all the arguments that will be used to execute the action method, which makes them a powerful way of extracting common logic out of your actions.

On top of this, they run after the action method has executed and can completely change or replace the `IActionResult` returned by the action if you want. They can even handle exceptions thrown in the action.

NOTE Action filters don't execute for Razor Pages. Similarly, page filters don't execute for action methods.

The ASP.NET Core framework includes several action filters out of the box. One of these commonly used filters is `ResponseCacheFilter`, which sets HTTP caching headers on your action-method responses.